

INFO 531: Data Warehousing and Analytics In the Cloud
Ripa Shah
Week 3 takeaways and assignment
TEMPORAL DATA UPDATE METHODOLOGIES FOR
DATA
WAREHOUSING

The business managers and executives would depend on the data warehouse to make a strategic and tactical decision.

The accuracy of information depends upon the detailed data and time-varying data. The temporal aspect normally consists of valid time and transaction time. The valid time reflects the time duration when a particular tuple is true in modeled reality whereas transaction time reflects when a particular tuple is captured in the database.

Of course, the operational database only manages current data whereas the data warehouse data manages a large volume of the historical data. Temporal data warehouse provide a history of serialized changes of data identified by time when changes occurred wherein it includes querying current states as well as past states of a record.

Data warehouses are refreshed at the certain time interval with data from different operational databases. We do full refresh as well as incremental refresh at certain interval. Data warehouse stores data by using two aspects: (1) Staging subject areas : one to one relation between staging subject area and data warehouse (2)

Analytical subject area: Analytical subject area requires to collect data from more than one table or more than one staging subject area.

Temporal data has many issues including consistent aggregation in presence of time varying data, temporal queries of multi dimensional data, storage method and temporal view materialization. As I have mentioned earlier, ETL tool has been

utilized to implement a data load from staging tables. Once data load has been completed from the staging tables within two to four hour window. It will load the main data warehouse analytical subject areas. To overcome performance issue, we should use indexing. The row effective date columns may have partitioned primary index (PPI) defined on them. That will make queries faster as the partition primary index pulls rows based on partition number instead of a full table scan. When a query is run with filters on PPI columns the DBMS will directly pull data based on particular bucket(s) instead of scanning the whole table.

Figure 1:

Rahman

Temporal Data Update Methodologies for Data Warehousing

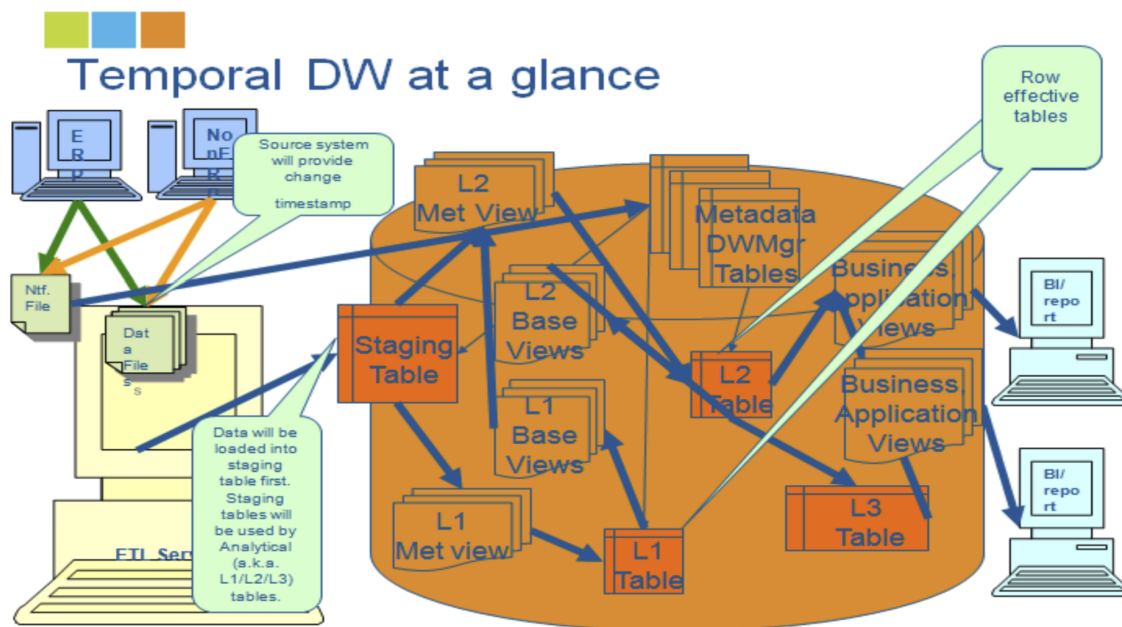


Figure 1: A Typical Temporal Data Warehousing Environment.

Figure 1 indicates the typical temporal data warehousing environment.

The record inserted in the data warehouse never deleted from the data warehouse to show the transaction lineage of that particular transaction. Moreover, it updates the data by expiring the existing row and inserting a new row. To implement

temporal data update technique, we require four columns including: 'row effective date', 'row effective time', 'row expired date' and 'row expired time'. The data manipulation (DML) code (I/U/D) and data row change

timestamp are provided by the source system during the refresh cycle. Also, if multiple version have the same primary key, previous version will be expired with the row effective timestamp of the immediate next version of that row. Only the current version will have the expired timestamp value '9999-12-31 12:00:00'.

After the staging load, if multiple version of data have been arrived in a file, SQL will process and expired the rows. These temporal data are useful in different fields including banking, retail sales, financial services, medical records, inventory management, telecommunications and reservation systems.

Figure 2:

■ Staging Table Refresh

Staging Table_A						
src_dml_cd	row_eff_dt	row_eff_tm	row_expr_dt	row_expr_tm	doc_nbr (PK)	amount
U	2013-10-12'	14:35:00'	9999-12-31'	00:00:00'	10002	\$12,000.00
U	2013-10-12'	15:19:00'	9999-12-31'	00:00:00'	10002	\$20,000.00
U	2013-10-12'	16:15:00'	9999-12-31'	00:00:00'	10002	\$23,000.00
D	2013-10-12'	17:25:00'	9999-12-31'	00:00:00'	10005	\$8,000.00
I	2013-10-12'	17:30:00'	9999-12-31'	00:00:00'	10006	\$8,000.00

■ Analytical Table Refresh

Analytical Table_A							
src_dml_cd	row_eff_dt	row_eff_tm	row_expr_dt	row_expr_tm	doc_nbr (PK)	amount	xfrm_asof_ts
I	2013-10-12'	10:05:00'	9999-12-31'	00:00:00'	10001	\$3,000.00	2013-10-12 14:00:00'
I	2013-10-12'	10:05:00'	9999-12-31'	00:00:00'	10002	\$10,000.00	2013-10-12 14:00:00'
U	2013-10-12'	14:35:00'	9999-12-31'	00:00:00'	10002	\$12,000.00	2013-10-12 18:00:00'
U	2013-10-12'	15:19:00'	9999-12-31'	00:00:00'	10002	\$20,000.00	2013-10-12 18:00:00'
U	2013-10-12'	16:15:00'	9999-12-31'	00:00:00'	10002	\$23,000.00	2013-10-12 18:00:00'
I	2013-10-12'	10:05:00'	9999-12-31'	00:00:00'	10003	\$6,000.00	2013-10-12 14:00:00'
I	2013-10-12'	10:05:00'	9999-12-31'	00:00:00'	10004	\$23,000.00	2013-10-12 14:00:00'
D	2013-10-12'	10:05:00'	9999-12-31'	00:00:00'	10005	\$8,000.00	2013-10-12 14:00:00'
I	2013-10-12'	17:30:00'	9999-12-31'	00:00:00'	10006	\$8,000.00	2013-10-12 18:00:00'

Figure 3: Table showing data after Incremental Refresh followed by Full Refresh.

Above figure 2 is an example of update with incremental data.

Temporal data must be evaluated by join with inequality predicates; rather than equality predicates. The join facilitates the one to one relationship should there be lower and upper bounds of the timestamp intervals that are not the same in the joining tables. The dimension table may have different lower and upper bounds of timestamps for current record, compared to primary source or fact table, as dimension data changes slowly.

Figure 3:

The figure 3 explains how joins works in data warehouse

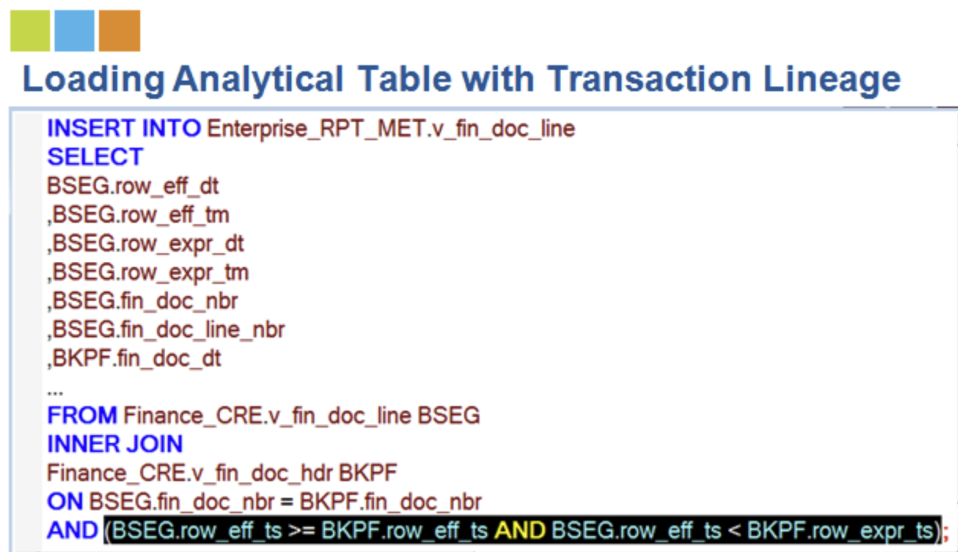


Figure 4: Temporal Relation between primary source and dimension tables.

The business users would like to see only the consistent data of current refreshed cycle that happened across all the tables in the subject area. They rather not be interested in viewing appended and changed data in a particular table. Source application business views will be defined on top of base views (for dirty reads). These views will provide data with filter based on application needs

```

REPLACE VIEW Capital_Analysis.v_fact_fin_doc_line
AS
SELECT *
FROM Capital_DRV.v_fact_fin_doc_line BSEG
WHERE ((BSEG.row_eff_dt = DATE '2013-10-12' AND BSEG.row_eff_tm <= TIME '08:15:30')
      OR (BSEG.row_exp_dt < DATE '2013-10-12'))
AND BSEG.row_exp_dt > DATE '2013-10-12';

```

Figure 6: Filters to pull rows up to a particular cycle refresh.

```

REPLACE VIEW Capital_Analysis.v_fact_fin_doc_line
AS
SELECT *
FROM Capital_DRV.v_fact_fin_doc_line BSEG
WHERE ((BSEG.row_eff_dt = DATE '2013-10-12' AND BSEG.row_eff_tm >= TIME '08:15:30')
      OR (BSEG.row_exp_dt > DATE '2013-10-12'))
AND ((BSEG.row_eff_dt = DATE '2013-10-13' AND BSEG.row_eff_tm <= TIME '08:20:31')
      OR (BSEG.row_exp_dt < DATE '2013-10-13'));

```

Figure 7: Filters to pull rows based on a particular time slice.

Data quality is also important for any business success to capture accurate profit, sales etc. During the load process, from source data to staging area and also from staging area to analytical area, utmost care must be taken to avoid data corruption.

Data quality might be compromised due to following reasons:

- (1) While joining multiple source tables, if join operators do not consider inequality predicates the integrity of timestamp-based data might be compromised.
- (2) Data quality might be compromised due to lack of maintaining referential integrity
- (3) Data quality also might be compromised while retrieving data from analytical tables viewing information via reporting, business intelligence, and data mining tools. This could happen due to missing join criteria or bad transformation logic.

The accuracy of information is dependent upon accuracy of the data and time variant data. The

